AI Tool Usage

Crypto Advisor — Moveo Coding Assignment

Author: Roy Meoded

AI tool used: Claude Code

Document date: August 2026

Overview

I used Claude Code (Anthropic's AI coding assistant) throughout this assignment — for requirement analysis, architecture and database design, implementation of every backend and frontend feature, debugging, automated testing, and visual design work. I worked through the project phase by phase, reviewing each piece of generated code, running it, and verifying it with real evidence (live HTTP requests, a real PostgreSQL database, and browser automation) before moving on. I remained responsible for every architectural decision, every accepted or rejected suggestion, and the final security and correctness of the implementation.

Tools Used

Tool	Main use
Claude Code	Repository-wide implementation: backend (FastAPI/SQLAlchemy/Alembic), frontend (Next.js/React/TypeScript/Tailwind), test authoring and execution, live verification via Playwright browser automation, and iterative visual design.

How AI Was Used

Requirements and planning

Claude Code broke the Moveo assignment into an implementation order (foundation → database → auth → onboarding → crypto data → AI insight → dashboard → feedback → deployment) and flagged which pieces were mandatory versus optional, so scope stayed aligned with the original assignment PDF rather than expanding into unrequested infrastructure (no Kubernetes, no multi-agent system, no vector database — all explicitly out of scope for this MVP).

Architecture and database design

Proposed the four-entity schema (`User`, `UserPreference`, `DailyInsight`, `ContentFeedback`) with the uniqueness and check constraints the assignment specifies, and the route → service → client layering used throughout the backend. I reviewed the autogenerated Alembic migration before applying it.

Authentication

Implemented signup/login/JWT and the protected-route dependency. When `passlib` (originally proposed) crashed against `bcrypt 5.0.0`'s internal self-test, I had Claude Code drop `passlib` entirely and hash passwords with the `bcrypt` library directly — verified by re-running the auth test suite.

Onboarding and personalization

Implemented the 3-step questionnaire and the atomic "save preferences + mark onboarding complete" transaction, and the dashboard section-reordering logic driven by saved `content_types`.

External API integrations

Implemented the CoinGecko and CryptoPanic clients with timeouts, caching, and labeled fallback behavior. During live testing, the default OpenRouter model (`openai/gpt-oss-20b:free`) turned out to have moved to OpenRouter's paid tier — caught via a real `404` response, not assumed — so I had it check OpenRouter's public model list and switch the default to a model still confirmed free.

AI insight feature

Implemented the grounded prompt (real fetched prices/news only, no invented market facts), the one-insight-per-user-per-day persistence rule, and the safe non-persisted fallback. Live testing against a real OpenRouter account also hit a real `429` rate limit; the app correctly returned the labeled fallback instead of crashing, which confirmed the entire error path live.

Feedback system

Implemented the upsert-based thumbs-up/down system with stable content keys. While reviewing test results in this session, I found that `test_request_body_cannot_select_another_user` failed — not from a code regression, but because the test queried `content_feedback` by `content_key` alone, with no `user_id` filter, and collided with real feedback rows already present from my own actual use of the deployed-locally app. I fixed the test to scope its query by `user_id`, which is what it was actually meant to check.

Dashboard and UX / visual design

Built the four dashboard sections with independent loading/error states so one failing provider never breaks the others. Later in the project, I worked with Claude Code through several rounds of visual design: a consistent mascot illustration set (generated with Google Gemini, not scraped), a light/dark theme system, a split auth layout, and a scrolling "coin marquee" component. Two representative fixes from that work:

- A CSS animation bug where the marquee's counter-rotation only canceled the ring's motion, not each badge's own placement angle, leaving every badge frozen at a tilt — fixed with a per-badge CSS custom property.
- A theme-toggle hydration mismatch, where the server always renders the light-mode icon but the client's first render already reflects the real (possibly dark) theme. The first fix attempt (a `mounted` flag set in a `useEffect`) tripped an ESLint rule against synchronous `setState` in effects; the actual fix used `useSyncExternalStore`, React's dedicated primitive for values allowed to differ between server and client render.

Testing and debugging

Backend and frontend test suites were built alongside each feature, not after. Provider-client tests are mocked at the `httpx` transport level so they never depend on live third-party services; database-

backed tests use a per-test transaction rollback fixture. All verification in this session was done by actually running the suites and inspecting real output, not by reading code and assuming it works.

Deployment

Not yet reached — see the main README.

Representative Interactions

The examples below are representative summaries, not a complete transcript.

Area	Request	AI contribution	My decision and verification
Auth	Fix a bcrypt/passlib crash on signup.	Identified passlib as unmaintained and incompatible with bcrypt 5.0.0; proposed hashing directly with bcrypt.	Adopted the change; re-ran the auth test suite to confirm signup/login still worked.
AI insight	Diagnose a live 404 from OpenRouter for the configured model.	Queried OpenRouter's public model list and identified a still-free replacement model.	Verified the new model's free status directly against OpenRouter's docs before adopting it, rather than trusting the suggestion blindly.
Design	Integrate a pasted third-party UI snippet (a radial background component).	Rewrote it to read the project's own theme CSS variables instead of hardcoded colors, so it adapts to dark mode automatically.	Verified in the browser in both light and dark mode before accepting.
Design	Integrate a pasted 3D marquee snippet with fake customer testimonials.	Flagged that fabricated testimonials shouldn't be used, and reused the project's own real crypto coin names in the existing tested marquee component instead.	Reviewed and approved this substitution before implementation.
Testing	A previously-passing test started failing.	Traced the failure to real feedback data from the developer's own account rather than a code bug, and fixed the test's query to be properly scoped.	Confirmed by re-running the full backend suite (138 tests passing) after the fix.

Suggestions Accepted

- Dropping `passlib` in favor of the `bcrypt` library directly, once it was shown to be the actual cause of a live crash.
- Using `useSyncExternalStore` instead of an effect-based `mounted` flag for the theme toggle — a smaller, more correct fix than the first attempt.

- Deriving the "depth" illusion in the scrolling coin marquee (badges appearing larger/brighter at the front) from pure CSS animations rather than pulling in a 3D/canvas library, keeping the dependency surface small.

Suggestions Modified or Rejected

- **A literal 3D perspective marquee with fabricated customer reviews** (from a pasted third-party snippet) was rejected. Fabricated testimonials misrepresent the product, so I had it reuse the project's existing, tested 2D marquee populated with real cryptocurrency names instead.
- **Wholesale adoption of a separately-generated Lovable design** (a different, TanStack-based codebase) was rejected in favor of extracting only its color tokens and applying them inside the existing Next.js implementation — avoiding a risky framework swap for a purely visual improvement.
- **Adding a `cn` / `clsx` utility** (present in a pasted snippet) was rejected to keep the codebase consistent with its existing plain-template-literal className convention, since the project doesn't use that pattern anywhere else.
- **A full rotating 3D sphere of tag-cloud items** (matching a reference image exactly) was scaled back to a simpler flat-ring CSS animation with a synthetic depth effect — visually similar, without the cross-browser fragility of real 3D transforms for this use case.

Human Verification

I verified AI-assisted work by:

- Running the backend test suite (pytest) against a real PostgreSQL database — currently 138 tests passing.
- Running the frontend test suite (vitest) — currently 34 tests passing.
- Running `next build` and `tsc --noEmit` after every frontend change.
- Driving the app with Playwright through real signup → onboarding → dashboard flows, in both light and dark themes, checking for console/hydration errors on every change.
- Testing external-provider failure paths live (a real 404 from a retired OpenRouter model, a real 429 rate limit) rather than only through mocks.
- Reading OpenRouter's and CoinGecko's current documentation directly instead of relying on remembered API details.

Limitations and Responsible Use

AI-generated suggestions were not treated as correct by default — several were rejected or modified after review, as documented above. The dashboard's AI insight is informational content only, not financial advice, and is always shown with a fixed disclaimer. Feedback is stored for possible future

recommendation-model training, but this version does not train or fine-tune any model. Automated tests reduce, but do not eliminate, the chance of undiscovered bugs.

Final Responsibility

I am responsible for the final implementation and for every decision made in this project. Claude Code supported the development process, but I reviewed, tested, modified, and in some cases rejected its suggestions, and I understand how every part of the resulting system works.

> This document accompanies the Crypto Advisor submission for the Moveo coding assignment. It reflects an honest account of AI-assisted development and is not itself part of the application's runtime behavior.